MODUL

LKS PROVINSI JAWA TIMUR







CLOUD COMPUTING

2025

Description of Project and Tasks

This module is **Deployment Automation and CI/CD.**

You are a DevOps engineer at Kivotos Tech Co., a fast-growing tech company that has recently scaled its development team from 10 to 50 engineers. With this growth, the number of projects has also increased significantly, leading to challenges in project management and coordination. The team is struggling to track tasks, deadlines, and progress across multiple projects, resulting in missed deadlines and inefficient workflows.

To address this, the CTO has decided to implement a project management tool. However, due to strict compliance requirements, the company cannot use public SaaS solutions like Jira or Trello. Building a custom project management app from scratch is also not feasible, it would take too much time and cost too much.

After evaluating the options, the CTO and development team have decided to use self-hosted open-source project management tools. Your task is to ensure the applications are deployed and monitored effectively so that the team can work without interruptions. Additionally, you need to ensure that the solution is secure and compliant, as the company handles sensitive user data.

Task

1. Read the documentation thoroughly (Outlined below).
2. Prepare and log in to your GitHub Account.
3. Log in to the AWS Academy Lab Console.
4. Please read and understand the ***Application Details*** section carefully**.**
5. Set up VPC configurations following the instructions in the ***Network - Service Details*** section.
6. Set up security groups following the instructions in the ***Security - Service Details*** section.
7. Set up a relational database. You can read the ***Database - Service Details*** section.
8. Set up a network file system using EFS. You can read the ***File System - Service Details*** section.
9. Set up the SSM parameter store. You can read the ***SSM Parameter Store - Service Details*** section.
10. Set up the container registry following the instructions in the ***ECR - Service Details*** section.
11. Define the Application Load Balancer and Auto Scaling for the container services. You can read the ***High Availability - Service Details*** section.
12. Set up the ECS cluster. You can read the ***ECS - Service Details*** section.

13. Configure GitHub Actions to define the CI/CD pipeline. You can read the ***Pipeline Details*** section.
14. Verify the applications.

Technical Details

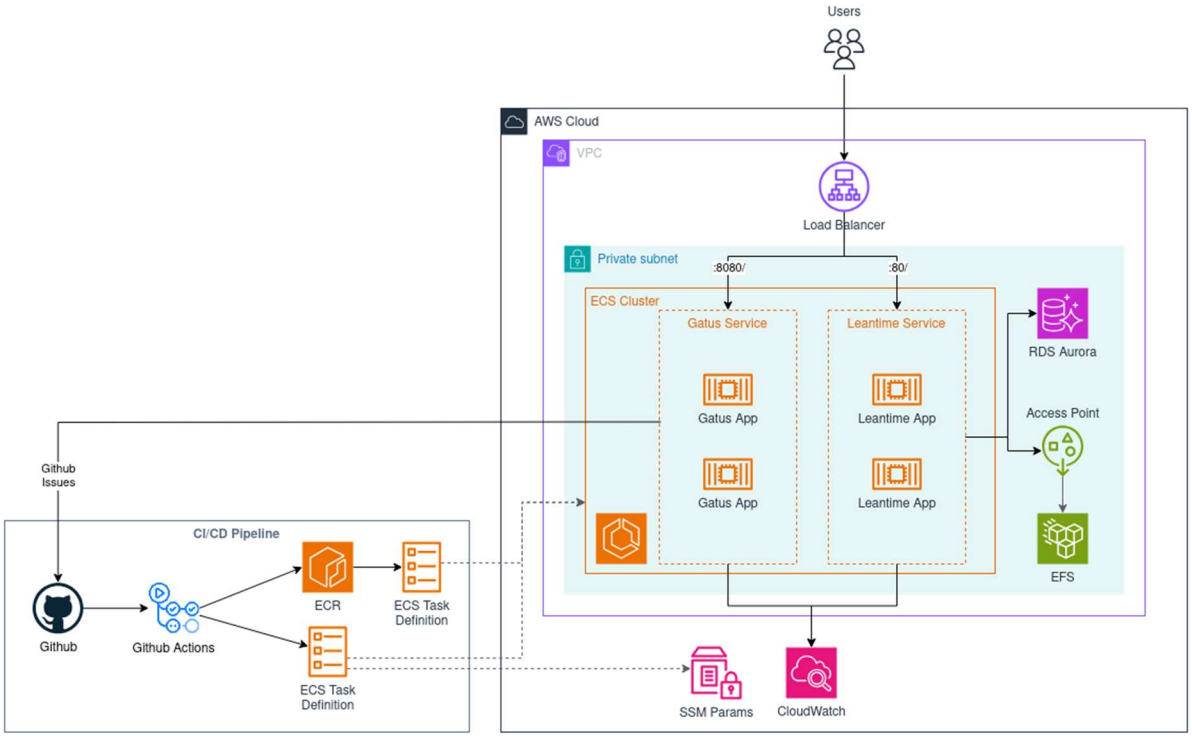
1. The goal of this project is to deploy an open-source project management application called Leantime and a status page called Gatus until they are fully operational.
2. The template source code is available on GitHub will be informed.
3. The default region is **N. Virginia**.
4. All the services that require an IAM role must use the **LabRole** role.
5. Every service you have created must use the "**lks-**" prefix for the naming format (e.g., lks-pm-vpc, lks-oss-cluster, etc.), unless explicitly instructed not to use it.
6. Label/tag each AWS service that you create, except for those that were automatically generated.
7. Remove unnecessary items or services to keep your work clear for easy assessment by the jury and avoid potential point deductions.
8. Use a local code editor such as VS Code, Vim, Nano, or Emacs to do your work without any extensions/plugins installed.

References

1. For documentation Leantime App, you can access this link: https://docs.leantime.io/
2. For documentation Gatus, you can access this link: https://gatus.io/docs | https://github.com/TwiN/gatus?tab=readme-ov-file#table-of-contents
3. For documentation VPC, you can access this link: https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html.
4. For documentation on RDS, you can access this link: https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html.
5. For documentation EFS, you can access this link: https://docs.aws.amazon.com/efs/latest/ug/whatisefs.html.
6. For documentation ECS, you can access this link: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html.
7. For documentation ECR, you can access this link: https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html.
8. For documentation SSM Parameter Store, you can access this link: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html

9. For documentation Application Load Balancer, you can access this link: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html.
10. For documentation GitHub Actions, you can access this link: https://docs.github.com/en/actions/writing-workflows/quickstart

Architecture



Application Details

There are two applications that need to be deployed. Both applications are open source web-based applications. First is Leantime, a project management application. Second is Gatus, a system monitoring dashboard to help monitor web pages and send notifications. The initial primary task is to configure the application through the use of environment variables or specific configuration files. The subsequent task entails defining a task definition to ensure the proper execution of the ECS service across all applications.

Leantime App

The Leantime application requires proper environment variable configuration to function correctly in your deployment environment. The sample env file is available in the repository located in the **leantime/** directory. Configure the application by setting the app URL and app port to 8080 to match your container configuration. Set the

sitename to **LKS Leantime** to properly brand the application. Configure the default timezone to **Asia/Jakarta** to ensure that all time-based features display correctly for your users. You are allowed to customize colors or themes according to your preferences, but ensure that any customizations work properly without breaking the application functionality. During development, you may enable debug mode to troubleshoot issues, but you must disable it before finalizing your deployment to ensure the application is production-ready and secure.

Define the ECS task definition for the Leantime application using the file in the repository located at **leantime/task-definition.json**. Complete any missing configuration as marked in the file. Set the family name to **lks-leantime-app**, and in the container definitions, map both the container port and host port to 8080 or match the app port defined in the environment variable. Configure the log driver to **awslogs** and set the log group to **/ecs/lks/leantime-app** so that logs are sent to AWS CloudWatch. Use **AWS VPC** for the network mode, restrict the task definition's **required compatibilities to Fargate**, and allocate 1024 CPU units along with 2048 MB of memory.

Add all necessary environment variables in the **environment property inside the container definition**. Any environment variables containing sensitive values such as passwords, secrets, and database credentials **must be stored and loaded from the SSM parameter store** to prevent security vulnerabilities that can lead to point deductions (read more in the *Service Details – SSM Parameter Store* section to set up the parameter store). **Define volumes based on the EFS access points** and **attach them to container mount points** with the read-only settings disabled and following configurations:

EFS access point	Container path	Description
/public-files	/var/www/html/public/userfiles	Store public files such as the logo, etc.
/userfiles	/var/www/html/userfiles	Store the user uploaded files
/plugins	/var/www/html/app/Plugins	Store the plugin for the application

Gatus

This app does not require environment variables, instead, it has one YAML file to configure. It's located in **gatus/config/config.yaml**. The initial step entails the configuration of an alerting provider with **GitHub issues within the repository**. **DO NOT directly store the URLs and GitHub token** used to send the alert. This can be done via the environment variables in CI/CD using GitHub Actions. Failure to do so may lead to the vulnerability of the application being exposed, resulting in the **loss of any points earned**.

Create two endpoints for the Leantime app URL and the Gatus URL. Set the Leantime app URL **interval to 30 seconds**, and configure conditions used to determine the health of the endpoint with a **status code equal to 200**, and the **response time must be below 300ms**. For the Gatus URL, set the **interval to one minute**, and configure conditions used to determine the health of the endpoint with a **status code equal to 200**, and the **response time must be below 500ms**. Set up an alerting provider to GitHub with a **failure threshold of 3** and a **success threshold of 5**. **Enable send on resolved** settings to automatically close the issue when the problem is resolved. You can use the default description provided in the repository or create your custom description.

To define the ECS task definition for the Gatus application. The base task definition is available in the repository under **gatus/task-definition.json**. Complete any missing configuration marked within the file. Set the family name to **lks-gatus-app** and configure the container definitions by mapping both the container port and host port to 8080. Additionally, set the log driver to **awslogs** and configure the log group to **/ecs/lks/gatus-app** to ensure that logs are properly stored in AWS CloudWatch. Set the network mode to use **AWS VPC**, specify that the **required compatibilities property is only for Fargate**, and allocate 512 CPU units and 1024 MB of memory for the task.

Service Details

Network

Provision network infrastructure before deploying the applications in the specified region with CIDR 10.100.0.0/16. Configure VPC with the following specifications:

- VPC name must be l**ks-pm-vpc**;
- Public subnets start with CIDR 10.100.0.0/24 with a minimum of 2 availability zones;
- Private subnets start with CIDR 10.100.10.0/24 with a minimum of 2 availability zones;
- The Internet gateway name is **lks-pm-igw**;
- Separate route table for public and private routes. Name these route tables **lks-pm-public-rt** and **lks-pm-private-rt**, respectively;
- Attach 1 NAT Gateway and name it **lks-pm-natgw**.

Security

Security is the most important aspect to consider when building infrastructure. Create a security group for instance applications, **allowing inbound traffic only from the Application Load Balancer (ALB)**. This restricts access to the services, ensuring that they can only be accessed through the ALB, which acts as a gateway for incoming

traffic. The Application Load Balancer (ALB) has a **separate security group** that only exposes **HTTP ports 80 and 8080 from the public**.

Secure the database and file system by preventing public access, ensuring those resources are not accessible from the internet. **Connections to the database and file system are allowed only from the applications**, ensuring that access is restricted to authorized sources. These measures enhance the security posture of the environment, reducing the risk of unauthorized access and ensuring the integrity of the application and its data.

Database

The application uses MySQL to store user information. Create **RDS Aurora for MySQL** with MySQL 8.0 major version compatibility. Use the production template and set the cluster identifier name to **lks-project-management-cluster**. Choose Aurora Standard for the storage configuration. Enable Aurora replica and configure the instance with instance class **t4g.medium** to get the best price-performance database. Secure the database by preventing access from the public network and using an existing security group that was previously created. In the monitoring section configuration, use **standard database insight** and **disable enhanced monitoring**. You also need to set up the username, password, and **leantime_db** as the initial database name required for the application. Copy and save those database credentials for future use.

File System

The applications require a centralized file system to store all media files uploaded by users. You must configure the Elastic File System with the following specifications: give the name **lks-project-management**, the type is **Regional**, enable **automatic encryption** of data at rest to ensure data security, set the performance mode to **General Purpose**, and set the throughput mode to **enhanced, elastic**. Secure the file system by preventing access from the public network and using an existing security group that was previously created.

To facilitate the storage of user assets by the Leantime application, it is necessary to create access points for the **/public_userfiles**, **/userfiles**, and **/plugins** directories, respectively. Use the User ID and Group ID that are used in the Leantime environment variables for directory creation. Set permission **full access for the user, read and write for groups, and read only for others**, in accordance with the principle of least privilege. The access points are utilized by the container, which is mounted on the container via the ECS task definition. Read the *Application Details - Leantime App* section to mount EFS to the application instances.

SSM Parameter Store

The Leantime application requires a database to store data in AWS RDS. The database configuration data is retrieved from the SSM Parameter Store using the path **/lks/db/{{KEY}}**. The following keys must be created in the parameter store:

Key Name	Type	Description
host	String	Database host/endpoint
port	String	Database port
name	String	Initial/default database name
username	SecureString	Database username
password	SecureString	Database password

Elastic Container Registry (ECR)

ECR is used as a container image repository to store the Gatus application with its configuration. Create an ECR private repository with the name **"lks-gatus"** and make sure **the tag is mutable**. This is to keep the latest tag in application images up-to-date. Each image will be published automatically via CI/CD to this private registry with the following tags:

<account-id>.dkr.ecr.us-east-1.amazonaws.com/lks-gatus:**<commit-sha>**
<account-id>.dkr.ecr.us-east-1.amazonaws.com/lks-gatus:**latest**

High Availability

To ensure high availability and proper load distribution for your application, you need to configure a Target Group for the Gatus service. Create a target group with the name **lks-gatus-app-tg**, set the target type to IP address, configure the target port to 8080, and use HTTP/1.1 as the target protocol. For the health check settings, configure the following parameters to monitor the health of your service:

- Use HTTP as the health check protocol
- Set the health check path to the root path
- Configure a healthy threshold of 5 and an unhealthy threshold of 3
- Set the timeout period and check interval to one minute, and set 200 as the success code that indicates a healthy target.

For the Leantime service, you need to configure another Target Group to properly route and monitor traffic. Create a target group with the name **lks-leantime-app-tg**, the target type to IP Address, configure the port to 8080, and use HTTP/1.1 as the

protocol. For the health check settings, configure the following parameters to effectively monitor the health of your Leantime service:

- Use HTTP as the health check protocol.
- Configure a healthy threshold of 3, an unhealthy threshold of 2, a timeout period of 30 seconds, and a check interval of one minute.

When configuring the health check path and success code, you must account for the behavior of the Leantime application, which **redirects requests from the root path to /auth/login with a status code of 302**.

After creating the target groups for both services, set up an Application Load Balancer to distribute incoming traffic. Create an Application Load Balancer with the name **lks-oss-lb**, set **the scheme to internet-facing** to allow access from the internet, and configure the IP type as IPv4. Use the VPC that has already been provisioned and select the appropriate existing security group to control traffic access. Configure two listeners to route traffic to your services: create a listener on HTTP port 80 that forwards traffic to the **Leantime service target group**, and create another listener on HTTP port 8080 that forwards traffic to the **Gatus service target group**. This configuration will enable your users to access the Leantime application through the standard HTTP port while making the Gatus monitoring interface available on a separate port.

Elastic Container Service (ECS)

The application needs a container orchestration platform to manage and deploy your containerized services. You must create an ECS Cluster with the name **lks-oss-cluster** and **enable AWS Fargate (Serverless)** in the infrastructure section to eliminate the need for managing server infrastructure, allowing you to focus solely on deploying and running your containerized applications.

Within the ECS cluster you've already created, define an ECS Service to run the Leantime application. Configure the ECS Service with the following specifications:

- Compute configuration: Set the capacity provider strategy to Fargate, configure the base value to 0 and the weight to 1, and choose the latest platform version.
- Deployment configuration: Use the leantime task definition with its latest version, and name the service **leantime-service**.
- Network configuration: Use the private subnet from the VPC that is already configured, disable public IP assignment, and select the existing security group created for the application to enhance security.
- Load balancing configuration: Enable load balancing, and select the existing Application Load Balancer and target group configured for the Leantime application to ensure the tasks are distributed properly.
- Service Auto Scaling configuration: Enable auto scaling for the service with a minimum number of tasks of 1 and a maximum of 5. Use Target Scaling as the

scaling type. Configure the scaling policy based on CPU utilization at 75% and set both the scale-out and scale-in cooldown periods to 300 seconds.

Create a second ECS Service to run the Gatus monitoring application within your cluster. Configure this ECS Service with the following specifications:

- Compute configuration: Set the capacity provider strategy to Fargate, configure the base value to 0 and the weight to 1, and choose the latest platform version.
- Deployment configuration: Use the Gatus task definition with its latest version, name the service **gatus-service**, and set the desired number of tasks to 2.
- Network configuration: Use the private subnet from the VPC that is already configured, disable public IP assignment, and select the existing security group created for the application to enhance security.
- Load balancing configuration: Enable load balancing, and select the existing Application Load Balancer and target group configured for the Gatus application to ensure the tasks are distributed properly.

Pipeline Details

GitHub Token

The token is used to authenticate GitHub actions. Create Fine-Grained Tokens ONLY for your repository. In repository access set permissions to be able to r**ead/write on Issues, read on metadata**. Copy and save this token for future use.

Reference: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens#creating-a-fine-grained-personal-access-token

GitHub Secret

Add the secrets to your GitHub repository. The credentials that need to be added to the secret repository are as follows:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- AWS_SESSION_TOKEN
- ALB_ENDPOINT
- TOKEN

GitHub Actions Workflow

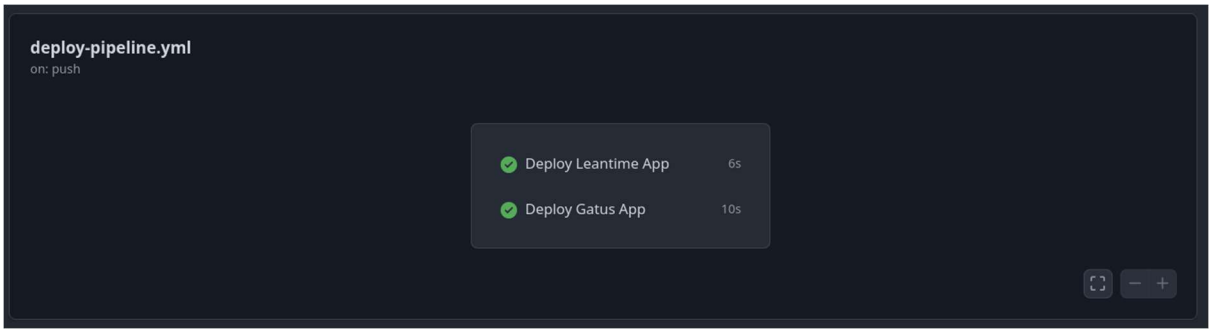
The workflow is used to automate the deployment to the ECS cluster. The workflow has multiple jobs running in parallel. Example jobs are available in the **.github/workflows/** directory in files with the **prefix “example-”**. Therefore, it is

necessary to complete or fix the steps and the workflow environment variables so that the jobs in the workflow can run properly. Steps that need to be filled in will be marked.

Create workflow files named **lks-deployment-pipeline.yml**. Set up workflow event triggers based on the code that is **pushed to the main branches** along specific paths **leantime/**, **gatus/**, and **.github/workflows/lks-deployment-pipeline.yml**. Each file has several jobs with the following details:

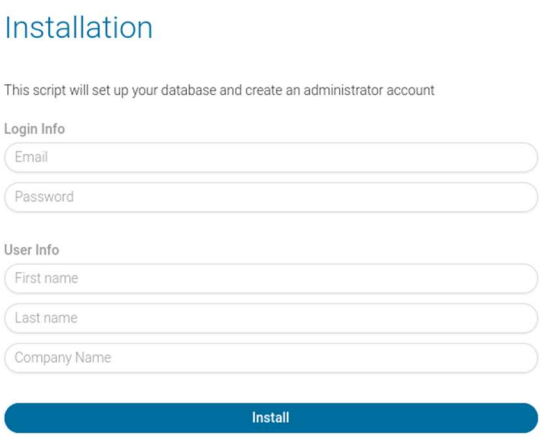
- **Deploy Leantime**: The job render the ECS task definition for the Leantime App and deploy it to ECS service.
- **Build Gatus**: The job performs configuration of applications, builds and tags an image, and publishes it to the ECR private registry. This job merges with the Deploy Gatus job.
- **Deploy Gatus**: The job configures and render the ECS task definition for the Gatus App and deploy it to ECS service.

Note: Make sure to delete all unused workflow files and only have the deployment workflow file in your repository. An example of a pipeline image on GitHub Actions is as follows:



Test and Verify Result

If the deployment is successful, you can access the applications via the ALB endpoint. The Leantime app will prompt you to set up the application.



If the system goes down, you'll be notified via a GitHub Issue in the repository.

